Converting lazy data frames into Parquet files (Python version)

Ian D. Gow

23 March 2026

Abstract In a note I published yesterday (*Converting lazy data frames into Parquet files*), I showed how my `db2pq` R package can be used to turn "lazy data frames" produced by the R package `dbplyr` into Parquet files on disk without needing to load the data into memory. Today, I wondered if I could do the same with my `db2pq` *Python* package. In this note, I turn "lazy data frames" produced by Ibis into Parquet files on disk.

> **Note**

The source for this note is available in a folder on GitHub that contains two files:

- `ibis_to_pq.qmd`
- `pyproject.toml`

To run this note locally with `uv`, put the two files above in a directory on your computer, I will refer to this directory as the **project directory** in this now.

For the convenience of *Tidy Finance* readers, I follow the approach laid out here closely. The *Tidy Finance* instructions start with installation of "`uv`, a modern Python package and project manager." You should install `uv` using the instructions provided by *Tidy Finance*; these are taken from the homepage for `uv`.

From the project directory, run the following command:

```
uv sync
```

Next, you should follow the instructions provided by *Tidy Finance* for setting things up to connect to WRDS:

> Create a `.env` file in your project directory. For the purpose of this book, we create and save the following variables (where user and password are our private login credentials):
>
> ```
> WRDS_USER=user
> WRDS_PASSWORD=password
> ```

Once you have done the above, you should be able to run the code in this note. For example, you could `uv run jupyter lab` and copy-paste the commands show here (in order). Alternatively, if you are comfortable with Quarto, you could open `ibis_to_pq.qmd` and treat it like a notebook.

1 Motivating example

According to Tidy Finance with Python:

> The daily CRSP data file is substantially larger than monthly data and can exceed 20 GB. This has two important implications: you cannot hold all the daily return data in your memory (hence it is not possible to save the entire dataset to your local folder), and in our experience, the download usually crashes (or never stops) because it is too much data for the WRDS cloud to prepare and send to your R session.
>
> There is a solution to this challenge. As with many big data problems, you can split up the big task into several smaller tasks that are easier to handle. That is, instead of downloading data about all stocks at once, download the data in small batches of stocks consecutively.

Below I show how one can simplify the code dramatically (no batches!) and download the data faster (for me, it takes *less than a minute*) and with no significant burden on either the CPU or RAM.

An issue with translating the `lazy_tbl_to_pq()` function from the R version of my `db2pq` package is that there is no equivalent of `dbplyr` for Python. While Python Polars has its `LazyFrame`, the Tidy Finance task involves extracting data from a PostgreSQL database and Polars has no way of creating lazy data frames from

database connections. The closest analogue is surely Ibis. So I added a function `ibis_to_pq()` to my Python package `db2pq` and in this note I take it for a spin.

I start by creating a connection to the WRDS PostgreSQL database. Here I follow the *Tidy Finance* approach closely. But so long as `wrds` is a `psycopg`-backed SQLAlchemy engine for the WRDS PostgreSQL database, you should be able to use whatever approach you are used to.

```
from sqlalchemy import create_engine
import os
from dotenv import load_dotenv

load_dotenv()

connection_string = (
    "postgresql+psycopg://"
    f"{os.getenv('WRDS_USER')}:{os.getenv('WRDS_PASSWORD')}"
    "@wrds-pgdata.wharton.upenn.edu:9737/wrds"
)

wrds = create_engine(connection_string, pool_pre_ping=True)
```

Having established `wrds`, I load in the packages I will be using.

```
import polars as pl
import ibis
from ibis import _
from db2pq import ibis_to_pq
```

Then I turn `wrds` into an Ibis instance with WRDS PostgreSQL as its **backend**. You can learn more about Ibis here.

```
db = ibis.postgres.from_connection(
    wrds.connect().connection.driver_connection
)
```

I then set up Ibis **lazy tables** for the three tables I will use here.[1]

```
dsf = db.table("dsf_v2", database="crsp")
stksecurityinfohist = db.table("stksecurityinfohist", database="crsp")
factors_daily = db.table("factors_daily", database="ff")
```

Ibis offers an "interactive" option that makes it easier to work with lazy tables.

```
ibis.options.interactive = True
```

Let's look at `ff3`:

```
factors_daily
```

[1]The R version of the note did not use `ff.factors_daily` from WRDS, but I do so here because I don't have the same `copy_inline()` functionality I had when using R in yesterday's note.

```
┌──────────────┬──────────────────┬──────────────────┬──────────────────┬──────────────────┬──────────────────┐
│ date         │ mktrf            │ smb              │ hml              │ rf               │ umd              │
├──────────────┼──────────────────┼──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ date         │ decimal(8, 6)    │ decimal(8, 6)    │ decimal(8, 6)    │ decimal(7, 5)    │ decimal(8,
6) │
├──────────────┼──────────────────┼──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ 1926-07-01 │     0.000900 │    -0.002500 │    -0.002700 │      0.00010 │         NULL │
│ 1926-07-02 │     0.004500 │    -0.003300 │    -0.000600 │      0.00010 │         NULL │
│ 1926-07-06 │     0.001700 │     0.003000 │    -0.003900 │      0.00010 │         NULL │
│ 1926-07-07 │     0.000900 │    -0.005800 │     0.000200 │      0.00010 │         NULL │
│ 1926-07-08 │     0.002200 │    -0.003800 │     0.001900 │      0.00010 │         NULL │
│ 1926-07-09 │    -0.007100 │     0.004300 │     0.005700 │      0.00010 │         NULL │
│ 1926-07-10 │     0.006100 │    -0.005300 │    -0.001000 │      0.00010 │         NULL │
│ 1926-07-12 │     0.000400 │    -0.000300 │     0.006400 │      0.00010 │         NULL │
│ 1926-07-13 │     0.004800 │    -0.002800 │    -0.002000 │      0.00010 │         NULL │
│ 1926-07-14 │     0.000400 │     0.000700 │    -0.004300 │      0.00010 │         NULL │
│ …          │            … │            … │            … │            … │            … │
└──────────────┴──────────────────┴──────────────────┴──────────────────┴──────────────────┴──────────────────┘
```

And then `dsf`:

```
dsf
```

```
┌────────┬────────────┬────────┬────────┬──────────────┬──────────────┬──────────────┬──────────────┬──────────────
│ permno │ hdrcusip   │ permco │ siccd │ nasdissuno │ yyyymmdd │ sharetype │ securitytype │
securitysubtype │ usincflg   │ issuertype │ primaryexch │ conditionaltype │ tradingstatusflg
│ dlycaldt    │ dlydelflg │ dlyprc           │ dlyprcflg │ dlycap           │ dlycapflg │
dlyprevprc      │ dlyprevprcflg │ dlyprevdt   │ dlyprevcap      │ dlyprevcapflg │ dlyret
│ dlyretx          │ dlyreti          │ dlyretmissflg │ dlyretdurflg │ dlyorddivamt    │
dlynonorddivamt │ dlyfacprc       │ dlydistretflg │ dlyvol           │ dlyclose        │
dlylow          │ dlyhigh         │ dlybid          │ dlyask          │ dlyopen         │
dlynumtrd │ dlymmcnt │ dlyprcvol       │ dlycumfacpr      │ dlycumfacshr     │ cusip      │
ticker     │ exchangetier │ shrout │
├────────┼────────────┼────────┼────────┼──────────────┼──────────────┼──────────────┼──────────────┼──────────────
│ int32  │ string(8) │ int32  │ int32 │ int32       │ int32     │ string(3) │ string(4)
│ string(3)        │ string(1) │ string(4)   │ string(1)    │ string(3)        │ string(1)
│ date         │ string(1) │ decimal(13, 6) │ string(2) │ decimal(13, 2) │ string(2) │
decimal(13, 6) │ string(2)     │ date       │ decimal(13, 2) │ string(2)     │ decimal(10,
6) │ decimal(10, 6) │ decimal(10, 6) │ string(2)       │ string(2)     │ decimal(13, 6)
│ decimal(13, 6)   │ decimal(10, 6) │ string(2)      │ decimal(14, 0) │ decimal(13, 6) │
decimal(13, 6) │ decimal(13, 6) │ decimal(13, 6) │ decimal(13, 6) │ decimal(13, 6) │ int32
│ int16     │ decimal(14, 1) │ decimal(17, 12) │ decimal(17, 12) │ string(8) │ string(5)
│ string(3)     │ int32  │
├────────┼────────────┼────────┼────────┼──────────────┼──────────────┼──────────────┼──────────────┼──────────────
│  10000 │ 68391610  │   7952 │  3990 │      10396 │ 19860107 │ NS        │ EQTY         │
COM            │ Y         │ ACOR       │ Q           │ RW             │ A               │
1986-01-07 │ N           │        2.562500 │ BA         │         9430.00 │ BP         │
NULL │ NS             │ NULL        │            NULL │ MP             │            NULL │
NULL │           NULL │ NS            │ MR           │       0.000000 │        0.000000 │
```

```
1.000000 | NO           |          1000 |          NULL |          NULL |          NULL
|       2.375000 |       2.750000 |          NULL |      NULL |        9 |         2562.5
|  1.000000000000 |  1.000000000000 | 68391610  | OMFGA     | SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860108 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A              |
1986-01-08 | N          |        2.500000 | BA         |         9200.00 | BP         |
2.562500 | BA            | 1986-01-07 |        9430.00 | PB            |      -0.024390
|       -0.024390 |        0.000000 | NA             | D1            |        0.000000
|         0.000000 |        1.000000 | NO             |          12800 |           NULL
|           NULL |           NULL |       2.375000 |       2.625000 |           NULL |
NULL |        9 |        32000.0 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860109 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A              |
1986-01-09 | N          |        2.500000 | BA         |         9200.00 | BP         |
2.500000 | BA            | 1986-01-08 |        9200.00 | PB            |       0.000000
|        0.000000 |        0.000000 | NA             | D1            |        0.000000
|         0.000000 |        1.000000 | NO             |           1400 |           NULL
|           NULL |           NULL |       2.375000 |       2.625000 |           NULL |
NULL |        9 |         3500.0 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860110 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A              |
1986-01-10 | N          |        2.500000 | BA         |         9200.00 | BP         |
2.500000 | BA            | 1986-01-09 |        9200.00 | PB            |       0.000000
|        0.000000 |        0.000000 | NA             | D1            |        0.000000
|         0.000000 |        1.000000 | NO             |           8500 |           NULL
|           NULL |           NULL |       2.375000 |       2.625000 |           NULL |
NULL |       10 |        21250.0 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860113 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A              |
1986-01-13 | N          |        2.625000 | BA         |         9660.00 | BP         |
2.500000 | BA            | 1986-01-10 |        9200.00 | PB            |       0.050000
|        0.050000 |        0.000000 | NA             | D3            |        0.000000
|         0.000000 |        1.000000 | NO             |           5450 |           NULL
|           NULL |           NULL |       2.500000 |       2.750000 |           NULL |
NULL |       10 |        14306.3 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860114 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A              |
1986-01-14 | N          |        2.750000 | BA         |        10120.00 | BP         |
2.625000 | BA            | 1986-01-13 |        9660.00 | PB            |       0.047619
|        0.047619 |        0.000000 | NA             | D1            |        0.000000
|         0.000000 |        1.000000 | NO             |           2075 |           NULL
|           NULL |           NULL |       2.625000 |       2.875000 |           NULL |
NULL |       10 |         5706.3 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860115 | NS        | EQTY        |
```

```
COM            | Y         | ACOR      | Q          | RW             | A               |
1986-01-15 | N          |        2.875000 | BA         |        10580.00 | BP         |
2.750000 | BA            | 1986-01-14 |       10120.00 | PB            |       0.045455
|        0.045455 |        0.000000 | NA              | D1            |        0.000000
|        0.000000 |        1.000000 | NO             |           22490 |           NULL
|           NULL |           NULL |       2.750000 |       3.000000 |           NULL |
NULL |       10 |        64658.8 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860116 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A               |
1986-01-16 | N          |        3.000000 | BA         |        11040.00 | BP         |
2.875000 | BA            | 1986-01-15 |       10580.00 | PB            |       0.043478
|        0.043478 |        0.000000 | NA              | D1            |        0.000000
|        0.000000 |        1.000000 | NO             |           10900 |           NULL
|           NULL |           NULL |       2.875000 |       3.125000 |           NULL |
NULL |       10 |        32700.0 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860117 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A               |
1986-01-17 | N          |        3.000000 | BA         |        11040.00 | BP         |
3.000000 | BA            | 1986-01-16 |       11040.00 | PB            |       0.000000
|        0.000000 |        0.000000 | NA              | D1            |        0.000000
|        0.000000 |        1.000000 | NO             |            8470 |           NULL
|           NULL |           NULL |       2.875000 |       3.125000 |           NULL |
NULL |       10 |        25410.0 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|  10000 | 68391610  |   7952 |  3990 |      10396 | 19860120 | NS        | EQTY        |
COM            | Y         | ACOR      | Q          | RW             | A               |
1986-01-20 | N          |        3.000000 | BA         |        11040.00 | BP         |
3.000000 | BA            | 1986-01-17 |       11040.00 | PB            |       0.000000
|        0.000000 |        0.000000 | NA              | D3            |        0.000000
|        0.000000 |        1.000000 | NO             |            1000 |           NULL
|           NULL |           NULL |       2.875000 |       3.125000 |           NULL |
NULL |       10 |         3000.0 |  1.000000000000 |  1.000000000000 | 68391610  | OMFGA
| SC1          |   3680 |
|      … | …         |      … |     … |          … |        … | …         | …           | …
| …         | …          | …           | …               | …                | …          |
…         |               … | …          |               … | …         |               …
…            | …          |              … | …            |             … |             …
|               … | …              | …             |               … |               …
… | …              |               … |               … |               … |               …
|             … |              … |              … |         … |        … |             …
|               … |               … | …         | …         | …            |      … |
└──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──────────
```

```
start_date = "1960-01-01"
end_date = "2024-12-31"
```

For clarity, in translating the original *Tidy Finance* query, I break out the `security` sub-query from the main query.

```
security = (
    stksecurityinfohist
    .filter(
        _.sharetype == "NS",
        _.securitytype == "EQTY",
        _.securitysubtype == "COM",
        _.usincflg == "Y",
        _.issuertype.isin(["ACOR", "CORP"]),
        _.primaryexch.isin(["N", "A", "Q"]),
        _.conditionaltype.isin(["RW", "NW"]),
        _.tradingstatusflg == "A",
    ).select(
        "permno",
        "secinfostartdt",
        "secinfoenddt",
    )
)
```

I next construct the main `crsp_daily` query.

```
crsp_daily = (
    dsf
    .filter(_.dlycaldt.between(start_date, end_date))
    .inner_join(security,
                dsf.permno == security.permno)
    .filter(_.dlycaldt.between(_.secinfostartdt, _.secinfoenddt))
    .select(
        permno=_.permno,
        date=_.dlycaldt,
        ret=_.dlyret,
    )
    .drop_null(["permno", "date", "ret"])
    .left_join(
        factors_daily
        .select("date", "rf")
        .rename(risk_free="rf"),
        _.date == factors_daily.date,
    )
    .mutate(
        ret_excess=ibis.greatest(_.ret - _.risk_free, -1)
    )
    .select(
        "permno", "date",
        ret=_.ret.cast("float64"),
        ret_excess=_.ret_excess.cast("float64"),
    )
)
```

Let's take a peek at `crsp_daily`, which is still a lazy table:

```
crsp_daily
```

```
┌────────┬────────────┬──────────┬────────────┐
│ permno │ date       │ ret      │ ret_excess │
├────────┼────────────┼──────────┼────────────┤
│ int32  │ date       │ float64  │ float64    │
├────────┼────────────┼──────────┼────────────┤
│  10000 │ 1986-01-08 │ -0.024390 │  -0.024690 │
│  10000 │ 1986-01-09 │  0.000000 │  -0.000300 │
│  10000 │ 1986-01-10 │  0.000000 │  -0.000300 │
│  10000 │ 1986-01-13 │  0.050000 │   0.049700 │
│  10000 │ 1986-01-14 │  0.047619 │   0.047319 │
│  10000 │ 1986-01-15 │  0.045455 │   0.045155 │
│  10000 │ 1986-01-16 │  0.043478 │   0.043178 │
│  10000 │ 1986-01-17 │  0.000000 │  -0.000300 │
│  10000 │ 1986-01-20 │  0.000000 │  -0.000300 │
│  10000 │ 1986-01-21 │  0.000000 │  -0.000300 │
│      … │ …          │         … │          … │
└────────┴────────────┴──────────┴────────────┘
```

The next step is to create a Parquet file from `crsp_daily` using `ibis_to_pq()`.

```
%%time
ibis_to_pq(crsp_daily, "crsp_daily.parquet")
```

```
CPU times: user 2min 18s, sys: 11 s, total: 2min 29s
Wall time: 3min 56s
```

```
'crsp_daily.parquet'
```

For some reason, `ibis_to_pq()` does not perform quite as well as its sibling `lazy_tbl_to_pq()` in the R version of `db2pq`.

```
%%time
ibis_to_pq(factors_daily, "factors_ff3_daily.parquet")
```

```
CPU times: user 109 ms, sys: 21.1 ms, total: 130 ms
Wall time: 785 ms
```

```
'factors_ff3_daily.parquet'
```

1.1 Using the data

I figured that it would be a little dull to do no more than simply create the Parquet files. But looking through *Tidy Finance with Python*, the only use of `crsp_daily` seemed to be in the section *Estimating Beta Using Daily Returns*.

There *Tidy Finance* say:

> We then create a connection to the daily CRSP data, but we don't load the whole table into our memory. We only extract all distinct `permno` because we loop the beta estimation over batches of stocks with size 500. To estimate the CAPM over a consistent lookback window while accommodating different return frequencies, we adjust the minimum required number of observations accordingly. Specifically, we require at least 1,000 daily returns over a five-year period for a valid estimation. This threshold is consistent with the monthly requirement of 48 observations out of 60 months, given that there are roughly 252 trading days in a year.

The *Tidy Finance* code uses pandas, but I will use Python Polars, which does much better with larger data sets. I start by creating `LazyFrame` instances of the two tables that are used in calculating betas.

```
crsp_daily = pl.scan_parquet("crsp_daily.parquet")
factors_ff3_daily = pl.scan_parquet("factors_ff3_daily.parquet")
```

The next step is to implement the approach described above. Python Polars can handle the details of creating the windows and so on, but we don't really want to have to hand over all the data to Statsmodels to do the regressions.[2] But if you're reading *Tidy Finance with Python*, it seems reasonable to assume that you know how to calculate a univariate regression coefficient, so let's just use Polars expressions and do it by hand.

Unfortunately, it turns out that we will be calculating about 2.5 million betas over windows of up to about 1,250 days each and that will be a lot of data even for Polars![3]

So I ended up using batches of 500, just as *Tidy Finance* do. The trick is to keep using rolling windows, but to define them over **month labels** rather than over raw daily dates. Each daily observation is assigned to its month, and the rolling window then spans the current month and the prior 59 months. This gives a 60-month lookback while still ensuring that a stock-month appears in the output only if the stock actually traded in that month.

Tidy Finance do not provide any indication of how long their code takes to run, but let's see how long this takes to run:

```
%%time
min_obs = 1000
batch_size = 500

def get_betas_batch(permno_batch: pl.DataFrame) -> pl.LazyFrame:
    daily = (
        crsp_daily
        .join(permno_batch.lazy(), on="permno", how="semi")
        .select("permno", "date", "ret_excess")
        .join(
            factors_ff3_daily.select(
                "date",
                pl.col("mktrf").cast(pl.Float64),
```

[2]The original pandas code uses `smf.ols()` from Statsmodels.

[3]There will be a lot of overlap in these windows and I had thought that Polars would just take the data and just process the windows at the start of each month as it passed through for each `permno`, but it seems it literally stacks copies of data for each window and I quickly ran out of RAM when I tried this.

```
                pl.col("rf").cast(pl.Float64),
            ),
            on="date",
            how="inner",
        )
        .select(
            "permno",
            "date",
            "ret_excess",
            mkt_excess=pl.col("mktrf") - pl.col("rf"),
        )
        .with_columns(month=pl.col("date").dt.truncate("1mo"))
    )

    firm_months = (
        daily
        .select("permno", "month")
        .unique()
    )

    return (
        daily
        .sort("permno", "month", "date")
        .group_by_dynamic(
            "month",
            every="1mo",
            period="60mo",
            offset="-59mo",
            group_by="permno",
            closed="left",
            label="right",
        )
        .agg(
            beta=pl.cov("mkt_excess", "ret_excess") / pl.col("mkt_excess").var(),
            mean_ret=pl.col("ret_excess").mean(),
            mean_mkt=pl.col("mkt_excess").mean(),
            n=pl.len(),
        )
        .with_columns(month=pl.col("month").dt.offset_by("-1mo"))
        .join(firm_months, on=["permno", "month"], how="semi")
        .filter(
            pl.col("n") >= min_obs,
        )
        .with_columns(
            date=pl.col("month"),
            alpha=pl.col("mean_ret") - pl.col("beta") * pl.col("mean_mkt")
        )
        .select("permno", "date", "beta", "n", "alpha")
        .sort("permno", "date")
```

```
    )

permnos = (
    crsp_daily
    .select("permno")
    .unique()
    .collect()
)

betas = pl.concat(
    (
        get_betas_batch(permnos.slice(i, batch_size)).collect()
        for i in range(0, permnos.height, batch_size)
    ),
    rechunk=True,
).sort("permno", "date")
```

```
CPU times: user 32.3 s, sys: 15.9 s, total: 48.2 s
Wall time: 19.2 s
```

So about 15-20 seconds for the whole lot.

And let's take a peek at the data:

```
betas
```

permno	date	beta	n	alpha
i32	date	f64	u32	f64
10001	1989-12-01	0.091415	1005	0.001083
10001	1990-01-01	0.09536	1027	0.001044
10001	1990-02-01	0.098356	1046	0.001014
10001	1990-03-01	0.09657	1068	0.001004
10001	1990-04-01	0.094286	1088	0.000986
…	…	…	…	…
93436	2024-08-01	1.592899	1258	0.002095
93436	2024-09-01	1.595383	1258	0.002201
93436	2024-10-01	1.605146	1258	0.001993
93436	2024-11-01	1.62253	1258	0.002191
93436	2024-12-01	1.631224	1258	0.00221

1.2 The same analysis with Ibis and DuckDB

An obvious alternative is to let DuckDB do the rolling-window work directly. Ibis makes this straightforward: we can connect to its default DuckDB backend, query the Parquet files in place, and then get the result as Polars table.

Here I use the same idea as above. I label each daily row with its month, define rolling windows over those month labels, and then keep only the first daily row for each `permno`/month. The window spans the current month and the prior 59 months, which seems closest to the pandas implementation.

```
duck = ibis.duckdb.connect()
```

```
%%time
min_obs = 1000

crsp_daily = duck.read_parquet(
    "crsp_daily.parquet",
    table_name="crsp_daily")

factors_ff3_daily = duck.read_parquet(
    "factors_ff3_daily.parquet",
    table_name="factors_ff3_daily",
)

daily = (
    crsp_daily
    .select("permno", "date", "ret_excess")
    .join(
        factors_ff3_daily.select(
            "date",
            mkt_excess=(
                _["mktrf"].cast("float64") - _["rf"].cast("float64")
            ),
        ),
        "date",
    )
    .mutate(month=_.date.truncate("M"))
)

w_beta = ibis.trailing_range_window(
    ibis.interval(months=59),
    group_by=_.permno,
    order_by=_.month,
)

w_month = ibis.window(
    group_by=[_.permno, _.month]
)

betas_duckdb = (
    daily
    .mutate(
        beta=(
            _.mkt_excess.cov(_.ret_excess, how="sample").over(w_beta)
            / _.mkt_excess.var(how="sample").over(w_beta)
```

```
        ),
        mean_ret=_.ret_excess.mean().over(w_beta),
        mean_mkt=_.mkt_excess.mean().over(w_beta),
        n=_.permno.count().over(w_beta),
        first_month_date=_.date.min().over(w_month),
    )
    .filter(_.date == _.first_month_date, _.n >= min_obs)
    .select(
        "permno",
        "month",
        "beta",
        "n",
        alpha=_.mean_ret - _.beta * _.mean_mkt,
    )
    .rename(date="month")
    .order_by("permno", "date")
    .to_polars()
)
```

```
FloatProgress(value=0.0,
layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

```
CPU times: user 2min 33s, sys: 6.49 s, total: 2min 40s
Wall time: 14.7 s
```

And here are the resulting betas:

```
betas_duckdb
```

permno	date	beta	n	alpha
i32	date	f64	i64	f64
10001	1989-12-01	0.091415	1005	0.001083
10001	1990-01-01	0.09536	1027	0.001044
10001	1990-02-01	0.098356	1046	0.001014
10001	1990-03-01	0.09657	1068	0.001004
10001	1990-04-01	0.094286	1088	0.000986
…	…	…	…	…
93436	2024-08-01	1.592899	1258	0.002095
93436	2024-09-01	1.595383	1258	0.002201
93436	2024-10-01	1.605146	1258	0.001993
93436	2024-11-01	1.62253	1258	0.002191
93436	2024-12-01	1.631224	1258	0.00221